

Site Copilot

Agentic RFI and Daily Report assistant for construction jobsites.

Arsen Khanguieldyan

arsen.khanguieldyan@gmail.com

Portfolio piece for Suffolk Construction

TRY IT LIVE

Live demo suffolk-construction.up.railway.app

Source code github.com/smartpneucontact-sketch/Sufflk

Model Anthropic Claude Sonnet 4.6 · ~\$0.05 per RFI triage

SUMMARY

Site Copilot is a working portfolio demo built on the same stack Suffolk's AI Studio runs on. Two production-shaped AI agents — RFI triage and Daily Construction Report drafting — run on Claude Sonnet 4.6 with Agentic RAG over a synthetic project corpus. Every component has a 1:1 production target: FastAPI stays the same, hybrid retrieval maps directly onto OpenSearch, JSONL telemetry ingests into Databricks bronze, and a Terraform sketch deploys the whole thing to AWS Bedrock + ECS Fargate. Built in a week. Live and demoable in a browser.

THE PROBLEM THIS SOLVES

On a Suffolk-scale project, project engineers field hundreds of RFIs. Industry baseline for response is around 9.7 days. End-of-shift, a superintendent spends 30 to 60 minutes turning sloppy field notes into a structured Daily Construction Report. Both are high-frequency, citation-heavy, and bound entirely by the project's own documents — the textbook shape of an Agentic RAG workflow.

WHAT IT SHIPS

AGENT 01

RFI Triage

Reads an incoming RFI, retrieves relevant spec sections and similar prior RFIs from the project corpus, drafts a citation-backed response, classifies urgency, estimates rough cost and schedule impact across multiple resolution scenarios, and flags whether escalation to the Structural Engineer of Record is required.

Why it matters: compresses the 9.7-day industry baseline to roughly 30 seconds and \$0.05 per RFI. Every claim is cited. Refuses to over-escalate when the spec is unambiguous and prior EOR rulings are already on

AGENT 02

Daily Construction Report

Takes raw, fragmented end-of-shift superintendent notes and produces a structured DCR — normalized crews, areas worked, equipment, deliveries, follow-ups, and an executive summary. Risks are bucketed into a fixed taxonomy (safety, schedule, quality, weather, material, coordination) so downstream BI can aggregate across the portfolio.

Why it matters: turns 30 to 60 minutes of nightly paperwork into a 30-second review. Open RFIs in the notes are auto-extracted; weather forecasts are flagged against pending work.

HOW IT MAPS TO SUFFOLK'S NAMED STACK

Every component in this demo has a 1:1 production target. The swap from local demo to deployed AWS is mechanical, not architectural. The LLMClient interface mirrors Bedrock's Converse API, the VectorStore Protocol mirrors OpenSearch's kNN, and the trace JSONL format ingests directly into Databricks bronze with no transformation.

THIS DEMO	SUFFOLK PRODUCTION
Anthropic SDK + Claude Sonnet 4.6	AWS Bedrock — anthropic.claude-sonnet-4-6
BM25 (+ optional Chroma), fused via RRF	OpenSearch hybrid (BM25 + kNN)
FastAPI + Pydantic	FastAPI + Pydantic
JSONL traces on disk	CloudWatch Logs → Databricks bronze
Single-page HTML UI	Teams / SharePoint apps
GitHub Actions CI (lint + tests + eval gate)	Same
infra/terraform/ sketch (ECS Fargate)	Production Terraform / CloudFormation
Tool-use loop in agents/base.py	Bedrock Converse multi-tool loop

SAMPLE AGENT REASONING (REAL RUN)

Here is the actual tool sequence the RFI agent ran on the question 'Drawing shows column cover at 1-1/4 in., spec calls for 1-1/2 in., which governs?' Every retrieval, classification, and impact estimate is captured in the trace.

RUN_783C6865ED17 · 3 STEPS · 15,023 IN / 1,819 OUT TOKENS · \$0.072

```
step 0  retrieve(query="concrete cover requirements columns spec 03 30 00",
               source_type="spec")
        -> 7 chunks. top: spec:03_30_00_concrete $3.04 (score 8.36)

step 0  retrieve(query="column concrete cover conflict drawing vs spec",
               source_type="prior_rfi")
        -> 6 chunks. top: prior_rfi:rfi_0089 (score 16.01)

step 0  classify_urgency(is_safety=true, is_critical_path=true)
        -> urgency="high"

step 1  estimate_impact(scenario="catch before placement",
                       labor_hours=4, material_usd=50)
        -> cost=$430, delay=0d

step 1  estimate_impact(scenario="rebar already tied, must reset",
                       labor_hours=16, material_usd=200)
        -> cost=$1,720, delay=1d

step 2  final answer (cited): spec governs, 1-1/2 in. minimum.
```

ECONOMICS AT SCALE

Inference cost stays trivial even at portfolio scale. At current Sonnet 4.6 pricing and typical project RFI volume:

PER RFI \$0.05 real measured cost	PER SITE / WEEK ~\$2.50 at 50 RFIs/week	PER SITE / MONTH <\$15 Anthropic bill
--	--	---

LLMOPS

The role asks for monitoring latency, cost, adoption, and drift. The demo ships all four:

- Latency: every LLM call and tool call is wrapped in a span with `elapsed_ms`. p50 / p95 per agent is a one-line awk on the JSONL trace.
- Cost: per-call `cost_usd` is computed from per-million-token pricing in `llm.py`. Every run carries its cost in the response payload — visible in the UI telemetry strip.
- Adoption: each run carries `actor_id` and `source_surface` tags. Weekly active users per project and auto-resolution rate (share of RFIs the agent answered without human edit) are one query away.
- Drift: a rolling window watches three signals — response length, citation count, and escalation rate — and emits an alert on a 3-sigma move. Production target is Bedrock Evaluation or Databricks ML monitoring with a labeled baseline window.
- Eval gate: four RFI cases with expected behaviors live in `evals/rfi_cases.yaml`. CI runs them in mock-LLM mode on every PR; merges block on scorer failures.

WHY THIS APPROACH

Suffolk's AI Studio runs a 'walk the jobsite, ship MVPs in days, hand off to central engineering' playbook. Site Copilot is precisely the artifact that playbook produces: pick a workflow pain, build the agent against the project's own corpus, ship it on the central platform's stack, stand up the LLMOps spine, hand off cleanly. The two agents shipped here are the two most-cited use cases in Suffolk's hiring materials — RFIs and daily reports. The same loop trivially extends to lookahead planning, materials tracking, or submittal log review: new system prompt, maybe one new tool, ~80 lines of code.

WHAT IS INTENTIONALLY NOT BUILT

This is a one-week portfolio demo, not a production deploy. The README's gap list is direct:

- Corpus is synthetic but realistic. Real specs would need OCR, table extraction, and drawing-callout linking.
- Terraform deploys infra but no autoscaling, alarms, or multi-account boundaries.
- No Procore or OpenSpace integration yet. Procore for RFI/DCR submission and OpenSpace for photo metadata feeding the DCR risk surfacing are the natural next two integrations.
- Drift detector is the smallest thing that could plausibly trigger an alert. Production needs proper distribution tests (KS, PSI) against a labeled baseline.

IN ONE LINE

Working agentic AI on Suffolk's exact stack vocabulary, against Suffolk's most-cited use cases, shippable to a real jobsite in a sprint.